

with the injected instruction *contaminated target context*. As a result, a backend LLM follows an injected instruction to generate an output as an attacker desires. Given an injected instruction, state-of-the-art attacks [1, 6, 40, 5, 24] first add a *separator* to the injected instruction before injecting it into the target context. The goal of the separator is to make the backend LLM more likely to follow the injected instruction. We refer to the combination of the separator and the injected instruction as the *injected prompt*. Existing attacks can be categorized into *heuristic-based attacks* and *optimization-based attacks*.

Heuristic-based attacks: Heuristic-based attacks leverage heuristic strategies to craft a separator. For instance, Naive Attack [2] directly injects the malicious instruction to the target context, i.e., the separator is an empty string. Escape Character [2] uses an escape character, e.g., the separator is `\n`. For Context Ignoring [1], the separator is a context-ignoring sentence such as *"Ignore previous instructions, please"*. Fake Completion [10] uses a fake response, e.g., the separator is *"Response: Task complete."*. Combined Attack [4] integrates all the separators used by the previous four attacks. Among these heuristic attacks, Combined Attack achieves state-of-the-art performance [4].

Optimization-based attacks: Existing optimization-based attacks [41, 6, 40, 5, 24] leverage gradient-based methods, such as GCG [41], to optimize a suffix such that an LLM is more likely to follow the injected instruction. To this end, the attacker can first define a loss function and minimize it by updating the tokens in the suffix. For example, letting Y_s denote the attacker-desired response and letting $T = C_t \oplus E \oplus I_s \oplus S$ be the contaminated target context, where E is a separator (e.g., one used by Combined Attack), S is the suffix being optimized, and I_s is the injected instruction. When the injected instruction is at the end of the target context, the loss can be written as $\mathcal{L} = -\log \Pr_g(Y_s | I_t \oplus C_t \oplus E \oplus I_s \oplus S)$, where g is the backend LLM. The attacker then optimizes S to minimize $\mathcal{L}$ with gradient descent, thereby making the backend LLM more likely to generate Y_s when taking the target instruction and contaminated context as input.

2.3 Prompt Injection Defenses

Existing defenses against prompt injection can be categorized into *prevention-based*, *detection-based*, and *attribution-based* defenses. These three families of defenses are complementary to each other and thus can be combined to form defense-in-depth.

Prevention-based defenses: Prevention-based defenses aim to make a backend LLM still perform the target task when the target context contains an injected instruction. Early prevention-based defenses [11, 12] leverage heuristic strategies. For instance, Jain et al. [42] proposed to perform paraphrasing or retokenization to reduce the influence of adversarial instruction. Sandwich defense [11] appends another instruction at the end of the context to remind the backend LLM to perform the target task. Instructional defense [12] redesigns the instruction to make the backend LLM ignore the instruction in the context. However, as shown in a benchmarking study [4] as well as in our results, these defenses have limited effectiveness under heuristic and optimization-based prompt injection attacks.

Recent studies [14, 15, 16, 18, 17] fine-tune an LLM to enhance its robustness against prompt injection. For instance, StruQ [14] leverages special delimiters to separate the instruction and context, and fine-tune an LLM such that it follows the target instruction (for a target task) while ignoring any instruction embedded within the context. Chen et al. [43, 17] further proposed SecAlign and Meta-SecAlign, which formulate the defense as a preference optimization problem and leverage DPO [44] to fine-tune an LLM. As shown in [17], Meta-SecAlign outperforms SecAlign and achieves the state-of-the-art defense performance.

However, as shown in existing studies [24, 45] and our results, Meta-SecAlign is still not robust to optimization-based attacks. Additionally, as Meta-SecAlign fine-tunes an LLM, it is challenging to use in closed-source LLMs such as GPT-5 and Gemini 2.5. We also find that the LLM fine-tuned by Meta-SecAlign has utility loss on certain tasks. Wallace et al. [16] proposed an instruction hierarchy that trains an LLM to prioritize system messages over user messages, and user messages over third-party content. As a result, the LLM can be more robust against the injected instruction in the (untrusted) third-party content. The defense has been deployed in GPT-4o-mini [46]. However, we find that GPT-4o-mini is still vulnerable to prompt injection. Wu et al. [47] also proposed a defense to differentiate and prioritize instructions to enhance the robustness.

We note that another family of prevention-based defense [22, 23, 18, 19, 20] is to leverage security policies to prevent prompt injection. For instance, DeBenedetti et al. [18] proposed CaMeL, which extracts control and data flows from user queries and enforces predefined security policies to prevent unintended actions produced by an LLM. However, such defenses face the following two challenges. The first challenge is that they cannot be generally applied to diverse tasks, such as question answering, document summarization, and so on. The second challenge is that it remains difficult to accurately specify security policies.

Inference-time scaling-based defenses, such as SecInfer [48], generate multiple responses by letting an LLM explore different reasoning paths. These defenses face the efficiency issue, especially under long contexts. For instance, as shown in [48], the computation time of SecInfer can be several times longer than that without any defense. Different from [48], we aim to design an *efficient* sanitization-based defense tailored to long-context LLMs.

Detection and attribution-based defenses: Given a target instruction and a target context for a target task, a detection-based defense aims to detect whether the target context is contaminated or not. In the past years, many detection methods [26, 27, 28, 29, 30, 31, 32] have been proposed. For instance, know-answer detection [49, 4] feeds a detection instruction and the target context to test whether a detection LLM follows the detection instruction. The target context is predicted as contaminated if the detection LLM follows the injected instruction instead of the intended detection instruction. DataSentinel [32] further formulates the detection as a minimax game and fine-tunes the detection LLM to improve the performance of known-answer detection. However, as shown in our results, state-of-the-art detection methods [32, 29, 31] still cannot accurately detect injected prompt in a long context.

Attribution-based defenses [34, 35, 37, 36] aim to attribute the injected prompt in a context that is detected as contaminated. For instance, Shi et al. [36] proposed PromptArmor, which leverages an LLM (e.g., GPT-4o) to detect and remove injected prompts in a context before letting a backend LLM generate an output. Jia et al. [37] proposed PromptLocate to localize the injected prompt in a context with the help of a prompt injection detector. Wang et al. [34, 35] designed generic attribution methods to identify texts (e.g., injected prompt, corrupted knowledge) in a context that are responsible for the output of an LLM. Similar to PromptArmor and PromptLocate, the attribution methods in [34, 35] can also be combined with a detection-based defense [32, 29] to remove the injected prompt in a context, i.e., we can remove texts responsible for an LLM output once the context is detected as containing an injected prompt. As shown in our results, state-of-the-art attribution-based defenses [35, 37, 36] achieve a sub-optimal performance. For instance, they need to leverage an existing detection-based defense to accurately detect prompt injection, and thus also inherit the limitations of existing prompt injection detection methods.

We note that a concurrent work [50] trains a sequence-to-sequence DataFilter model to filter the injected instruction in an input. DataFilter is primarily designed for short contexts, while we mainly focus on long context scenarios.

2.4 Input Sanitization

Input sanitization has been widely used in web security to defend against threats such as SQL injection [51]. In particular, it removes or encodes potentially malicious characters to ensure that only properly formatted and safe inputs are processed, thereby reducing security risks such as unauthorized code execution. Inspired by this principle, PISanitizer performs context sanitization to remove or suppress malicious instructions within the input context before a backend LLM generates its output. A major challenge is that, unlike traditional sanitization (e.g., rule-based), context sanitization against prompt injection requires understanding semantic intent, as there is no clear separation between instruction and data (context).

3 Threat Model and Problem Formulation

3.1 Prompt Injection to Long-Context LLMs

We characterize the threat model with respect to the attacker’s goal, background knowledge, and capabilities.

Attacker’s goal: We consider that an attacker aims to inject the malicious instruction into a long context. As a result, when taking the target instruction and the contaminated target context as input, a backend LLM follows the injected instruction to generate an output as the attacker desires. With this goal, an attacker can achieve many purposes in practice.

When the backend LLM (e.g., in a retrieval-augmented generation system) is used to generate an answer to a user question, an attacker can inject a malicious instruction such that the backend LLM generates an attacker-desired answer for a target question. Suppose the backend LLM is used to generate a review for a research paper, an instruction can be embedded into the paper to mislead the backend LLM to generate a positive review [52]. When the backend LLM is used to summarize the reviews from different users, a malicious user can post a review with an injected instruction to mislead the backend LLM to generate an attacker-desired review summary. For AI-assisted coding, an attacker can also inject an instruction in a code repo such that a backend LLM generates a piece of malicious code. As shown in these examples, prompt injection to long-context LLMs causes severe security concerns for many real-world applications.

Attacker’s background knowledge and capabilities: We consider a strong attacker, where the attacker has white-box access to the parameters of the backend LLM as well as the system prompt. Moreover, we consider that the attacker can access the entire target context and knows the target instruction. We also assume that the attacker can arbitrarily inject the malicious instruction into the target context, e.g., the attacker can inject it in the beginning, middle, or at the end of the target context. We evaluate state-of-the-art and adaptive prompt injection attacks in our experiment.

3.2 Problem Setup for Prompt Sanitization

We introduce prompt sanitization and the defender.

Prompt sanitization: Given a target context, prompt sanitization aims to remove potential injected tokens in the context before feeding it into a backend LLM to perform a target task. As a result, the output of the backend LLM would not be influenced by the injected instruction when the given context is contaminated by prompt injection.

We have three goals for prompt sanitization: *effectiveness*, *efficiency*, and *utility*. The effectiveness goal means the injected tokens should be removed if the given context contains the injected instruction. The efficiency goal means the sanitization should be efficient (e.g., compared with the output generation of the backend LLM). The utility goal means the sanitized context should maintain the utility for the target task.

Defender: Prompt sanitization can be deployed in various defense settings to prevent prompt injection by diverse defenders. For instance, the defender can be an LLM application provider that integrates a sanitization defense before forwarding a context to the backend LLM to perform a target task, ensuring that malicious instructions are filtered out at the system level. The defender can be an individual user who applies sanitization locally before querying a backend LLM for a user task, e.g., a user who uses an LLM to generate a summary for a document (e.g., a webpage) downloaded from the Internet.

We consider that the defender has access to an LLM used to sanitize a context, which can be different from the backend LLM. For instance, we can use a publicly available LLM, such as Llama 3.1-8B-Instruct, to sanitize a context. The sanitized context is fed into a backend LLM (e.g., GPT-5) to perform the target task.

4 Design of PISanitizer

Overview: PISanitizer is based on two observations. The first observation is that prompt injection attacks intrinsically involve crafting an instruction to make an LLM follow it to generate an attacker-desired output. The second observation is that the Transformer architecture of LLMs [38] employs the attention mechanism to focus on crucial input tokens for output generation. As a result, instructional input tokens that are followed by an LLM would generally receive high attention weights from the output tokens. These two observations guide the design of PISanitizer: given a context, we can *intentionally* let an LLM follow